Direct Preference Optimization for Small Instruction Models

CS 555-455 Final Project Presentation

İde Melis Yılmaz & Hakan Mert Özcan
Faculty of Engineering and Natural Sciences, Sabancı University

2026

Outline

Why Align Language Models?

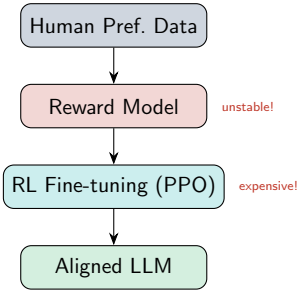
The Core Problem

We have a preference on LLM generated text in different contexts. Serious, detailed, fun...
But generally we want it to be helpful, honest and harmless.
Goal: Align model responses according to the human preferences.

Classical Pipeline: RLHF

1. Collect pairwise human preferences
2. Train a *reward model*
3. Fine-tune LLM with RL (PPO)

Problem: complex, unstable, expensive



Research Questions

1. Can DPO improve a small 0.5B language model?
2. How much does prompt engineering help?
3. Which preference optimization method works best?
   - DPO
   - IPO
   - ORPO
4. How should we evaluate alignment reliably?

Experimental Setup

Model

- Qwen2.5-0.5B-Instruct
- LoRA fine-tuning
- Only adapter weights updated

Dataset

- UltraFeedback (binarized)
- Chosen vs Rejected responses
- 2k and 5k training pairs

Evaluation

- Judge-free
- Log-probability ranking
- 1000-example test set

UltraFeedback Dataset

What is UltraFeedback?

- Large-scale instruction-following preference dataset
- Responses rated by GPT-4 across 4 dimensions: Instruction Following, Truthfulness, Honesty, Helpfulness
- **Binarized version (TRL-lib):** each pair has one *chosen* (higher-rated) and one *rejected* response
- We use deterministic splits: seed=42

Possible Bias

GPT-4 ratings used to create the dataset may themselves be biased toward longer, more verbose responses.

How Do We Evaluate Preferences?

Step 1: The model assigns probabilities to responses

Prompt:

"What is the capital of France?"

Preferred answer:

"Paris is the capital of France."

Rejected answer:

"France is a country in Europe."

The model computes:

$$p(\text{preferred}|x) \qquad \text{and} \qquad p(\text{rejected}|x)$$

Why use log probabilities?

- A sentence contains many tokens.
- Total probability is the product of token probabilities.
- Products become extremely small for long responses.
- Taking the logarithm turns multiplication into addition:

$$\log(ab) = \log a + \log b$$

- This is numerically stable and standard in language model evaluation.

Why Not Just Use Preferred = 1, Rejected = 0?

A dataset label only tells us:

$$\text{Preferred} > \text{Rejected}$$

It does **not** tell us:

- How strongly the model prefers one answer.
- Whether fine-tuning increased confidence.
- Whether two methods produce different preference margins.

Using log probabilities allows us to measure:

$$\text{Margin} = \log p(\text{preferred}|x) - \log p(\text{rejected}|x)$$

Larger margin = stronger alignment with human preference.

While exploring the dataset we noticed:

$$\text{Chosen answers} \approx 46 \text{ tokens longer}$$

than rejected answers.

Question:

Are our evaluation metrics biased?

Where Does Length Bias Come From?

Chosen responses are approximately **46 tokens longer** than rejected responses.

For a response with T tokens:

$$\text{Sum LogP} = \sum_{t=1}^{T} \log p(y_t|x)$$

Longer responses contain more terms in the sum.

$$10 \text{ tokens} \neq 100 \text{ tokens}$$

A model may appear better simply because it assigns probability to longer responses.

This can inflate preference metrics even when response quality does not improve.

Our Solution: Per-Token Normalization

Instead of using:

$$\sum_{t=1}^{T} \log p(y_t|x)$$

we also compute:

$$\frac{1}{T} \sum_{t=1}^{T} \log p(y_t|x)$$

This measures average confidence per token.

Benefits:

- Reduces response-length bias.
- Allows fair comparison between short and long answers.

Key Lesson: Always report both summed and length-normalized metrics.

Methods at a Glance

Method	Objective	Risk
DPO	Directly increase probability of preferred responses	Overfit to length
IPO	Target a fixed preference margin	Lower (robust)
ORPO	Remove reference model entirely	**Length shortcuts**

DPO

Simple, effective baseline. Most studied in literature.

IPO

More stable optimization. Better for implicit-reference LoRA setups.

ORPO

Most efficient. But there is a risk of bias.

Prompt Engineering Baselines

We tested **5 system prompt templates** at inference time (no retraining):

Name	PrefAcc(mean)	Description
Simple	0.5950	"Answer the user's question."
Structured	**0.5900**	6-criterion quality rubric (helpful, honest, faithful, clear, concise, complete)
Few-Shot	0.5850	"Imitate the style of high-quality examples"
CoT	0.5800	"Think through step by step internally, then answer"
Expert+Rubric	0.5850	Expert framing + full 6-point rubric

Finding

Results differ very little.So prompt engineering alone cannot substitute for parameter-level preference updates. Also structured prompt is used for the rest of the experiments.

Main Results — All Methods (n=1000, Structured Prompt)

Variant	PrefAcc(sum)	Margin(sum)	PrefAcc(mean/t)	Margin(mean)
Base (no fine-tuning)	0.444	−41.26	0.555	0.238
DPO 5k + Struc.	0.447	−40.68	0.558	0.244
IPO 5k + Struc.	**0.447**	−40.71	**0.562**	**0.248**
ORPO 5k + Struc.	0.446	−42.48	**0.551**	**0.200**

DPO

Modest consistent gains. Best "sum" metric improvement over base.

IPO - Winner

Highest per-token accuracy **and** margin.

ORPO

Worst per-token despite competitive sum.

Why Did IPO Perform Best?

Core Idea

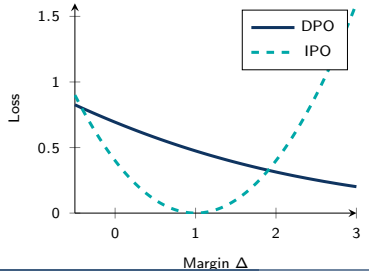
IPO does not continuously push the preference margin upward. Instead, it aims for a specific target margin. Both too-small and too-large margins are penalized.

$$\mathcal{L}_{IPO} = \left(\Delta - \frac{1}{2\beta}\right)^2$$

Why This Helps

IPO: **0.562**, DPO: 0.558, Base: 0.555

- Prevents excessive margin growth
- Less likely to exploit surface features
- Better generalization in low-data LoRA settings



Why ORPO Looked Good But Wasn't

ORPO

- No frozen reference model
- Alignment + language modelling together
- Most compute-efficient method

Our Discovery

Chosen responses in UltraFeedback are approximately

$$+46 \text{ tokens}$$

longer than rejected responses.
This creates a systematic length bias.

Evaluation Results

Method	Sum	Per-token
Base	0.444	0.555
ORPO	0.446	**0.551**

Interpretation

Sum metric improves which can be largely explained by response length. Per-token quality decreases.

Data Quality Ablation: Does Filtering Help?

Hypothesis: training on preference pairs with a larger score gap (high-margin) should give cleaner signal than random pairs.

Ablation Design (2k pairs)

- **Random**: uniform sample from UltraFeedback
- **High-margin**: pairs with largest preference score difference
- **Low-margin**: pairs with smallest score difference

Finding Why?

High-margin filtering **does not outperform** random sampling at 2k examples. At small data scales ($n = 2k$, 1 epoch, LoRA):

- High-margin pairs may have a narrow distribution — less diversity
- Statistical noise dominates signal from data filtering
- Method choice (DPO vs IPO) matters *more* than data curation at this scale

Why Are Gains So Small? (Understanding the Limits)

Model capacity

0.5B parameters is tiny. The model has limited room to re-rank preferred over rejected.

Data scale

2k–5k preference pairs, 1 epoch. Literature suggests DPO typically needs 10k–100k+ pairs for reliable gains. Our results are essentially a *low-data regime* experiment.

LoRA subspace constraint

LoRA freezes the base model and only adapts low-rank residuals. The representational changes possible are fundamentally limited — especially for learning deep preference patterns.

Note:

Despite small absolute gains, the **ranking** of methods is reliable: IPO $>$ DPO $>$ ORPO on per-token accuracy. This ordering is consistent across subsets and statistical tests.

Limitations

What we did NOT do

- Human evaluation
- Full model fine-tuning (only LoRA)
- Models > 0.5B

Evaluation limitations

1-All experiments on Qwen2.5-0.5B-Instruct. Results may not generalise to different architectures or pretraining regimes.

2-Judge-free log-prob ranking is reproducible and cheap, but:

- Length confounders even after per-token normalisation
- Correlation with human judgements is imperfect
- Tokenization and punctuation still affect margins

Future Work

- Larger datasets (10k–50k pairs)
- Larger models (7B+)
- **Targeted human evaluation** on cases where DPO and IPO disagree
- **LLM-judge cross-validation**: does log-prob ranking agree with GPT-4 judging?
- Scale training to 10k–50k pairs

Key Takeaways

1. **IPO achieves the highest per-token preference accuracy** among all methods tested.
2. **ORPO's reference-free formulation produces inflated sum-logP metrics** due to length effects.
3. **Sum logP metrics are biased by response length** (chosen $\approx +46$ tokens). Always report both sum *and* token-normalised variants.
4. **Method choice matters more than data filtering** at 2k training examples under LoRA. Invest in the right objective before investing in data curation.

Thank You!

Questions?

İde Melis Yılmaz • Hakan Mert Özcan

ide.yilmaz@sabanciuniv.edu

hakan.ozcan@sabanciuniv.edu

Backup: Full Results Table

Method	Prompt	n	PA(sum)	PA(mean)	Margin(sum)
Base	default	200	0.440	0.610	−51.21
Base	simple	200	0.440	0.595	−51.29
Base	structured	200	0.445	0.590	−51.07
Base	few-shot	200	0.430	0.585	−52.77
Base	CoT	200	0.445	0.580	−52.21
Base	exp.+rubric	200	0.435	0.585	−51.63
Base	structured	1000	0.444	0.555	−41.26
DPO 5k	structured	1000	0.447	0.558	−40.68
IPO 5k	structured	1000	0.447	0.562	−40.71
ORPO 5k	structured	1000	0.446	0.551	−42.48

Backup: LoRA — Why Not Full Fine-tuning?

LoRA (Low-Rank Adaptation):

$$W' = W_0 + \Delta W = W_0 + BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, rank $r \ll \min(d, k)$

- Only A and B are trained; W_0 is frozen
- At $r = 16$: $\approx 0.5\%$ of parameters updated
- Preserves base model; adapter can be swapped
- Fits in student-level GPU budgets

Trade-off

LoRA constrains parameter updates to a low-rank subspace. Behavioural shifts at small data scales are correspondingly limited — this explains (partly) why DPO gains are small.

Practical justification

Full fine-tuning a 0.5B model requires $\sim$4–8$\times$ the memory. Under course compute constraints, LoRA is the pragmatic choice.